

Google Cloud Agentic Cinema Hackathon

Partnerships Track • Rules Summary + Three Build Ideas

Contest window	July 27 – Sept 9, 2026 (2:00 PM PT close)
Judging window	On or about Sept 23 – Oct 7, 2026
Entry cost	Free – no purchase necessary
Total prize pool	\$75,000 across five identical partner tracks
Per-track prizes	1st \$7,500 • 2nd \$4,500 • 3rd \$3,000
Partner tracks	IBM • Grafana Labs • Parallel • ClickHouse • Replit
Required core stack	Gemini + Google Cloud Agent Builder / Agent Platform, plus your chosen partner's product or MCP server
AI tooling limit	Google Cloud AI tools only, plus your chosen partner's built-in AI features — no AWS, Microsoft, OpenAI or Anthropic AI tooling
Team size	Up to 4 people, one designated Representative
Platform	Web, Android, or iOS

The one-sentence brief

Build a working AI agent, powered by Gemini and Google Cloud Agent Builder, that plugs into one partner's product or MCP server to remove a real bottleneck for filmmakers, screenwriters, studio crews, or fans — then prove it runs, film a 3-minute demo, and open-source the repo.

Why “simple but powerful” wins here

Stage One is pass/fail on whether you actually met the requirements — a project that clearly and reliably does one thing beats an ambitious project that half-works. Stage Two then scores four equally weighted criteria (25% each), so a small, sharply-scoped agent that is genuinely useful, cleanly demoed, and visibly uses both Google Cloud and the partner service at runtime can outscore a sprawling one that doesn't finish. The three ideas in this brief are chosen for exactly that: one clear workflow, one partner integration, one believable user.

SUBMISSION CHECKLIST

What has to be true before you submit

Required in your Devpost submission

- A URL to the **hosted, working project** (must actually run, not just build).
- A **3-minute demo video** (public on YouTube or Vimeo) showing the project functioning as built — a screen-capture walkthrough, not a cinematic trailer. English or English subtitles.
- A **public, open-source repository** (GitHub / GitLab / Bitbucket) with all source code, assets, and run instructions, plus an **OSI-approved license file detectable in the repo's About section**.
- The repo must show **Google Cloud AI actually imported and called in code** (google-adk, google-genai, google-generativeai, or google-cloud-aiplatform), not just named in the README.
- The repo must show your **partner's SDK/MCP server actually running** at runtime — see the table below.
- A written description: features, tech used, data sources, and what you learned building it.
- Your chosen **partner track** selected on the submission form.
- The project must be **newly built during the contest period** — not a reskin of prior work.

Track-by-track: what “actually used it” means

Track	Runtime requirement that gets checked
IBM	Must be built using IBM Bob during development (this is checked). Confluent for real-time/event data is optional but encouraged.
Grafana Labs	Must actively call the Grafana Cloud MCP server (official grafana/mcp-grafana or the hosted endpoint) at runtime — not just AI Observability alone.
Parallel	Must actively call Parallel's Search API in code — e.g. the parallel-web SDK, the Vercel AI SDK tool, LangChain's ParallelWebSearchTool, or Grounding with Parallel as the provider.
ClickHouse	Must actively query a real ClickHouse Cloud or self-hosted cluster via the official mcp-clickhouse server at runtime.
Replit	Must be built with Replit Agent and the finished app must be hosted on Replit (a replit.app / replit.dev URL).

Eligibility note: entrants must be of majority age and not resident in a long list of embargoed / excluded countries and regions named in the Official Rules (including Italy, Brazil, Quebec, China, Russia, Iran, Cuba, North Korea, and others) — check Section 4 of the Official Rules if you are unsure your location qualifies.

JUDGING RUBRIC

How Stage Two is actually scored

Stage One is pass/fail: does the submission include every required item and reasonably use the required Google Cloud and partner products? Only projects that pass move to Stage Two, where four criteria are **equally weighted at 25% each**. Use the scorecard below to self-score a concept before you build it.

Criterion (25% each)	Judges are asking...	Score 1–5
Technological Implementation	How well is it built, and how effectively does it use Google Cloud and the partner service? Is the integration real and load-bearing, or bolted on?	■ ■ ■ ■ ■
Design	Is this a complete, coherent product experience — or just a technical proof of concept with no usable front end?	■ ■ ■ ■ ■
Potential Impact	Is there a credible, specific real problem for a real audience (filmmaker, crew, fan), and does the demo actually show it being solved?	■ ■ ■ ■ ■
Quality of the Idea	Is this a creative, non-obvious use of Google Cloud + the partner service, showing genuine understanding of the problem space?	■ ■ ■ ■ ■

Fast pre-flight check (Stage One, in plain terms)

- Does it run right now, from the hosted URL, without me setting anything up?
- Does the repo actually `import` and **call** a Google Cloud Gen AI SDK, not just mention it?
- Does the repo actually call the partner's SDK/MCP server at runtime, per the table on the previous page?
- Is the license file visible in the repo's About section on GitHub/GitLab/Bitbucket?
- Is the video ≤ 3 minutes, public, and does it show the thing actually working?
- Is this genuinely new work built during July 27 – Sept 9, 2026?

Script Doctor

A fact- and continuity-checking agent for screenwriters, built on Gemini + Parallel Search.

Writers and script supervisors burn real hours (and cause reshoots) chasing details a script casually asserts as fact: “this drug was legal in 1987,” “this radio model existed in 1965,” “this courthouse has jury trials on Saturdays.” Script Doctor reads an uploaded script or scene, has Gemini extract the checkable factual claims, sends each one to Parallel's Search API for grounded, sourced evidence, and returns a scene-by-scene accuracy report with citations and suggested fixes.

Who it's for	Screenwriters, script supervisors, and indie showrunners who can't afford a research department.
The loop	Upload/paste script → Gemini extracts factual claims per scene → each claim is queried through Parallel Search → Gemini reconciles results into a pass/flag/uncertain verdict with the source cited → report rendered scene-by-scene in the web UI.
Google Cloud	Gemini (via google-genai / Vertex AI) for claim extraction and verdict synthesis; optionally Agent Builder to structure the extract → search → verify pipeline as a defined agent flow.
Parallel usage	The parallel-web SDK calls Parallel's Search API for every extracted claim at runtime, with results passed back into Gemini — this is the load-bearing integration, not a side feature.
Why it's simple	One input type (a script), one linear pipeline (extract → search → verify → render), no multi-agent orchestration required to be credible.
Why it scores well	Technological Implementation: Parallel search directly drives the output, not decoration. Potential Impact: reshoots and factual continuity errors are a named, expensive, well-understood industry problem. Quality of idea: grounding a creative document in live web fact-checking is a genuinely non-obvious use of a search API.
Stretch (optional)	Let users click a flagged claim to see the actual Parallel source snippets, or export the report as production notes.

Location Scout AI

An agent that turns a scene description into a ranked, permit-aware location shortlist — built and hosted on Replit.

Independent filmmakers lose days to location scouting: matching a scene's mood and budget to a real place, then working out what permits and logistics it needs. Location Scout AI takes a plain-language scene brief (“rain-soaked alley, night, low budget, needs generator access”), has Gemini turn that into structured search criteria, evaluates candidate locations against those criteria, and returns a ranked shortlist with a permit/logistics checklist per option — all built with Replit Agent and deployed on Replit.

Who it's for	Indie directors, line producers, and student filmmakers without a locations department.
The loop	User describes the scene in natural language → Gemini structures it into criteria (mood, budget tier, access needs, permit sensitivity) → candidate locations are scored and ranked → Gemini drafts a logistics/permit checklist per location → results shown in a Replit-hosted web app.
Google Cloud	Gemini for criteria structuring, scoring rationale, and checklist generation, called via google-genai / Vertex AI from the app's backend.
Replit usage	Built end-to-end with Replit Agent as the primary development tool, and the finished app is deployed and hosted on a replit.app domain — satisfying the track's build-and-host requirement directly.
Why it's simple	No external data pipeline is required to be credible for a demo — a seeded set of sample locations with real attributes is enough to show the reasoning and ranking working end to end.
Why it scores well	Design: Replit's hosting makes a clean, immediately-usable product experience easy to achieve rather than a bare API. Potential Impact: location scouting is a concrete, unglamorous, expensive bottleneck that's easy for judges to picture. Technological Implementation: clean single-track use of Replit Agent for build + deploy, verifiable from the hosted URL alone.
Stretch (optional)	Wire in a real maps/places API for live candidate locations instead of a seeded set.

Production Pulse

A plain-English health-monitoring agent for render farms or live shoot infrastructure, powered by the Grafana Cloud MCP server.

VFX and post-production crews watch dashboards full of metrics, logs, and traces for render farms, streaming rigs, and on-set data infrastructure — but most of the crew isn't fluent in reading them under deadline pressure. Production Pulse is an agent that queries Grafana Cloud through its MCP server for the metrics, logs, and active alerts behind a production pipeline, and has Gemini turn that into a short, plain-English status brief: what's broken, what's at risk, and what to do next.

Who it's for	VFX supervisors, DITs, and technical producers who need a status read without parsing raw dashboards.
The loop	User asks (or a schedule triggers) a status check → the agent calls Grafana Cloud MCP tools to pull relevant metrics, logs, traces, and open alerts → Gemini synthesizes findings into a ranked, plain-English brief (what's failing, what's trending bad, suggested first action) → brief is shown in the app and can be pushed to the crew.
Google Cloud	Gemini (via google-genai / Vertex AI or Agent Builder) reasons over the structured data the MCP tools return and produces the natural-language triage summary.
Grafana usage	The agent connects to the official grafana/mcp-grafana server (or the hosted Grafana Cloud MCP endpoint) and actively calls its tools at runtime to query metrics, logs, traces, and alerts — the requirement the track explicitly checks for.
Why it's simple	One agent, one MCP connection, one output format (a status brief) — complexity lives in the Grafana data source you point it at, which can be a demo Grafana Cloud stack with seeded render-farm metrics.
Why it scores well	Technological Implementation: directly satisfies the track's explicit runtime check on the MCP server. Quality of idea: translating observability data into crew-readable triage is a non-obvious, genuinely useful reframing of a monitoring tool. Potential Impact: downtime on a render farm or live rig has a clear, quantifiable cost.
Stretch (optional)	Let the agent open or annotate a Grafana incident directly through the MCP server's alert-management tools, closing the loop from detection to action.

CHOOSING & NEXT STEPS

Which one to build

If you want...	Build
The easiest path to a hosted, working demo with minimal DevOps	Idea 2 – Location Scout AI (Replit)
The most novel, judge-catching use of a partner API	Idea 1 – Script Doctor (Parallel)
Comfort with observability data and a strong Technological Implementation story	Idea 3 – Production Pulse (Grafana Labs)

A simple build order that fits all three

- **Week 1:** stand up the Gemini call and the partner integration first, on fake/seeded data — prove the required runtime connection before building any UI around it.
- **Week 2:** wire in real inputs (a real script, a real seeded location set, a real demo Grafana stack) and build the minimal usable front end — this is what “Design” is scored on.
- **Before submitting:** re-run the Stage One pre-flight checklist on page 3, record the 3-minute demo video from the hosted URL, and confirm the license file shows in the repo's About section.

Sources

This brief is built entirely from the Google Cloud Agentic Cinema Hackathon – Google Cloud Partnerships Official Rules and the hackathon's public Devpost listing (agent-cinema.devpost.com), as supplied for this brief. Always confirm current details against the live Devpost page before submitting, as contest details can be updated.